

LangChain Integration

Auto-compress tool outputs in your LangChain.js agents. Cut tokens 30–70%, keep answer quality, ship in one line of middleware.

1 Why it matters

LangChain.js agents burn most of their tokens on *tool outputs* — search results, scraped pages, database rows — that the model rereads on every turn. Compresr replaces each raw output with a query-aware, compressed version *before* it enters the message state, so your KV cache, your prompt, and your bill all shrink. Drop it in, keep your agent exactly as is.

2 Install

```
npm install @compresr/sdk
export COMPRESR_API_KEY=cmp...
```

Node.js 20+. Dual ESM + CJS build; the `@compresr/sdk/integrations/langchain` subpath needs `langchain>=1.0.0` as a peer (and any `@langchain/*` packages you already use).

3 Drop-in agent middleware

One import, one middleware entry. The agent code below is unchanged from the LangChain.js docs except for the highlighted line.

```
import { createAgent } from 'langchain';
import { compresrToolMiddleware } from '@compresr/sdk/integrations/langchain';

const agent = createAgent({
  model: 'openai:gpt-4o-mini',
  tools: [webSearch],
  middleware: [
    compresrToolMiddleware({
      apiKey: process.env.COMPRESR_API_KEY!,
      queryArg: 'query',           // use args.query as the intent
      targetCompressionRatio: 0.5, // remove ~50% of tokens
      minTokens: 200,             // skip short outputs
    }),
  ],
});
```

That's it. The agent still sees every tool result, just shorter and focused on the current query. Fail-open by default: if the Compresr API is unreachable, the original output passes through and your agent keeps running.

4 Wrap a single tool

Prefer to compress just one noisy tool (e.g. a web scraper) inside an LCEL chain or LangGraph node? Use the wrapper — name, description, and schema are preserved.

```
import { wrapToolWithCompression } from '@compresr/sdk/integrations/langchain';

const compressedSearch = wrapToolWithCompression(rawSearch, {
  apiKey: process.env.COMPRESR_API_KEY!,
  queryArg: 'query',
  targetCompressionRatio: 0.5,
});
```

Also available: `compresrSummarizationMiddleware` (condenses conversation history at a token threshold) and `compresrPromptMiddleware` (enforces a hard prompt budget). For retriever-side compression, drop `CompresrExtractor` into a `ContextualCompressionRetriever`

```
import { CompresrExtractor } from '@compresr/sdk/integrations/langchain';
import { ContextualCompressionRetriever } from 'langchain/retrievers/contextual_compression';

const retriever = new ContextualCompressionRetriever({
  baseRetriever: vectorStore.asRetriever({ k: 10 }),
  baseCompressor: new CompresrExtractor({
    apiKey: process.env.COMPRESR_API_KEY!,
    targetCompressionRatio: 0.5,
  }),
});
```

5 Key parameters

Parameter	Default — Description
<code>apiKey</code>	<i>required</i> — your Compresr API key.
<code>targetCompressionRatio</code>	<code>0.5</code> — values in <code>[0, 1]</code> remove that fraction of tokens; values <code>> 1</code> are an <code>Nx</code> factor (e.g. <code>3</code> for <code>3x</code> shorter).
<code>minTokens</code>	<code>200</code> — skip outputs shorter than this. Keeps small results untouched and fast.
<code>queryArg</code>	<i>undefined</i> — name of the tool argument to use as the compression query (e.g. <code>'query'</code>).
<code>query</code>	<i>undefined</i> — static query string, used when every call shares the same intent.

Parameter	Default — Description
<code>queryExtractor</code>	<code>undefined</code> — custom function to derive a query. Middleware receives <code>(toolCall, messages) => string undefined</code> ; <code>wrapToolWithCompression</code> receives <code>(args) => string undefined</code> . Return <code>undefined</code> to fall back to the default.
<code>compressionModel</code>	<code>'latte_v1'</code> — query-aware default. Switch to <code>'latte_v2'</code> for the faster variant.
<code>coarse</code>	<code>undefined</code> — <code>true</code> for paragraph-level (faster), <code>false</code> for token-level (tighter).
<code>allowTools</code> / <code>ignoreTools</code>	<code>undefined</code> — limit compression to (or exclude) specific tool names.
<code>onError</code>	<code>'passthrough'</code> — fail-open behaviour. Set to <code>'raise'</code> to fail-closed.

Get started: `npm install @compresr/sdk` • Docs and pricing at compresr.ai.